Project 3 — Active Learning for Learning-to-Defer

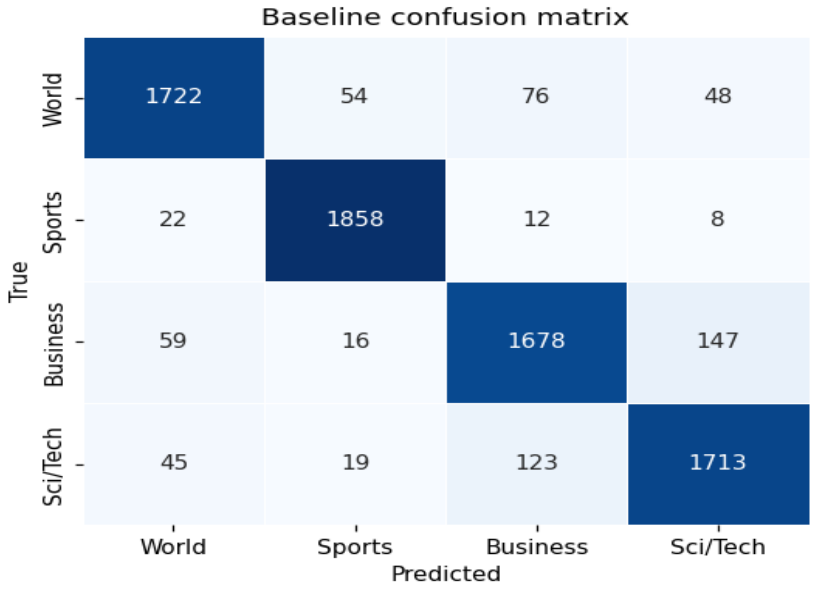
Human-Centric AI · TUHH · Summer semester 2026

This report documents the design choices and results for the five tasks of Project 3. The baseline classifier learns the AG News topic-classification task, a simulated expert models human specialisation on a subset of classes, a confidence-threshold policy decides when to defer to the expert, and an active-learning strategy discovers expert competence with a limited query budget.

Task 1 — Baseline classifier

A TF-IDF vectoriser (unigrams and bigrams, 20 000 features, sublinear TF weighting) followed by a multinomial Logistic Regression trained with L-BFGS on the full AG News training split (n=120000). Cached to disk after the first fit.

Test accuracy: 0.9172 on n=7600 examples.

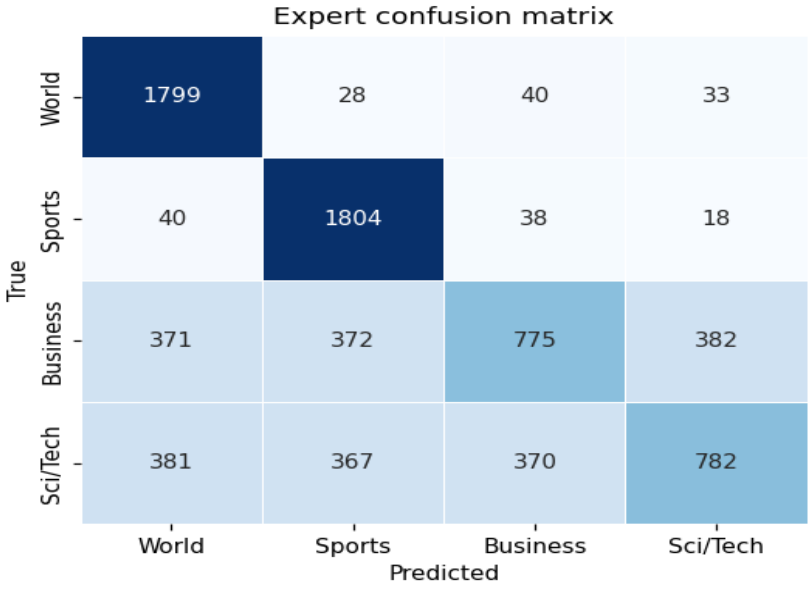


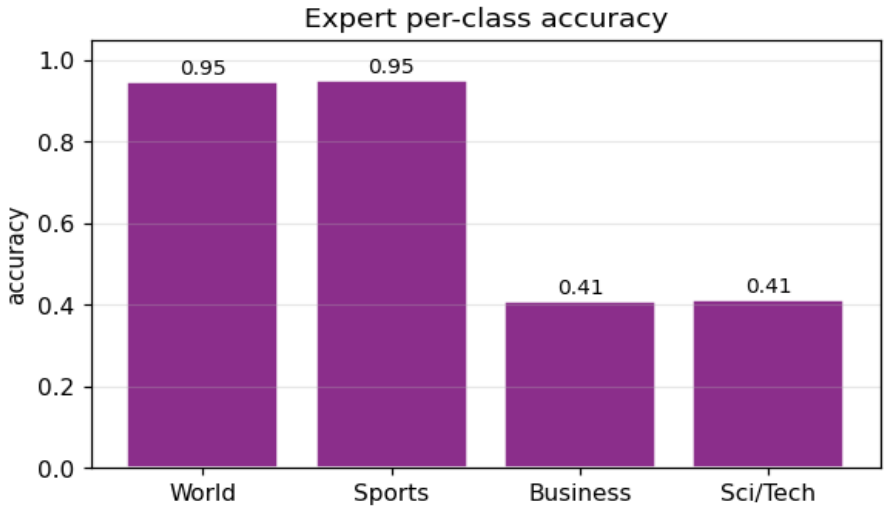
Class	Precision	Recall	F1	Support
World	0.9318	0.9063	0.9189	1900
Sports	0.9543	0.9779	0.9659	1900
Business	0.8883	0.8832	0.8857	1900
Sci/Tech	0.8941	0.9016	0.8978	1900

Task 2 — Simulated region-competent expert

Real human experts specialise. We model this with an expert that answers correctly with probability $\mathbf{p_c}$ on classes in its competence set and with lower probability $\mathbf{p_o}$ on the remaining classes. When wrong, a uniformly random incorrect label is returned. This gives the expert a competence *structure* that a deferral policy can later exploit.

Default configuration: competent on **World + Sports**, p_c=0.95, p_o=0.40 → overall accuracy **0.6789**.

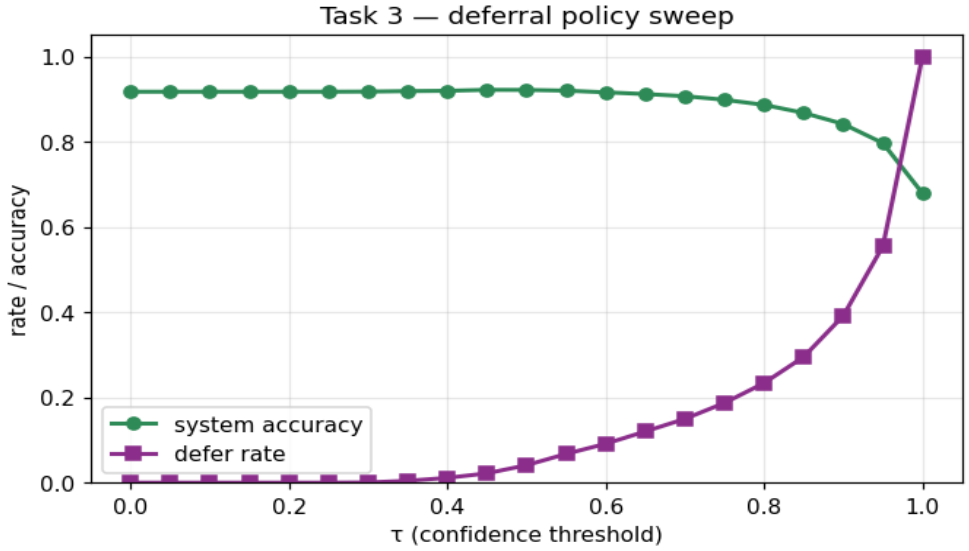




Task 3 — Learning to defer

Given the classifier's soft-max output and the (simulated) expert's label, we adopt the simplest well-motivated deferral rule: **defer to the expert whenever the top-class probability of the classifier is below** τ. The classifier's top-1 probability is a serviceable measure of its own confidence, so we defer precisely on cases where the classifier is unsure. When τ is small the system falls back to the classifier, when τ is large it always defers.

The plot below sweeps τ across [0, 1] and reports the system accuracy (a blend of model and expert predictions) together with the defer rate.



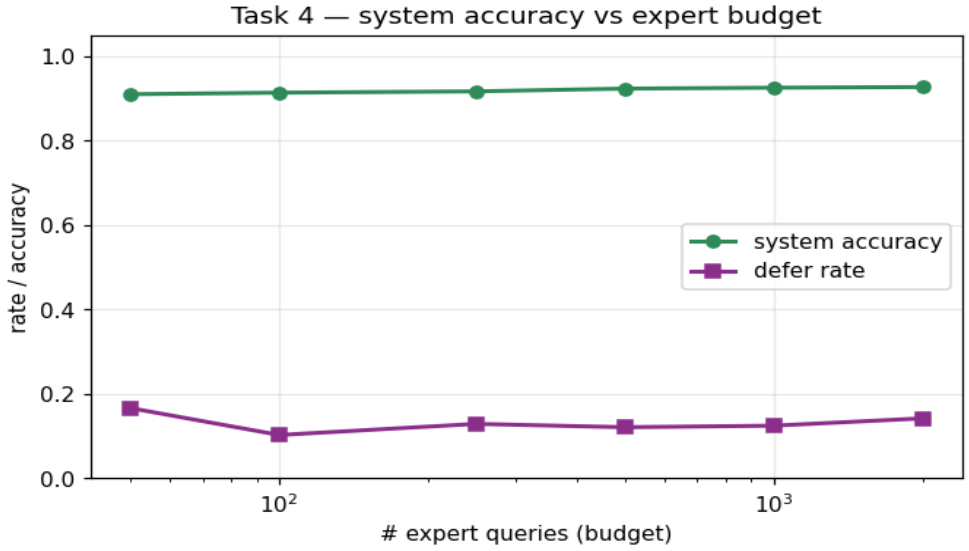
Because the expert is strong on two of the four classes and the classifier is uniformly strong (≈0.92 accuracy across all classes), the confidence-threshold policy only improves the system at very small deferral rates. Above $\tau \approx 0.6$ the system starts to lose accuracy because the expert is weaker than the classifier on Business and Sci/Tech, so deferring uncertain examples in those classes replaces good model predictions with noisy expert labels. The best operating point in the sweep is

τ **= 0.45** — defer rate 0.022, system accuracy **0.9216**.

Task 4 — Active learning for expert competence

We now assume no expert labels are available during training. Given a budget of B expert queries, we use **least-confident uncertainty sampling**: the B test points with the smallest max-softmax score are the ones where a deferral could plausibly help, so labels there give us the most information about whether the expert should be trusted.

From the queried labels we estimate the expert's accuracy conditioned on the model's *predicted* class (rather than the true class, which we do not know at deployment). At inference we defer whenever the estimated expert accuracy for the model's predicted class exceeds the model's top-1 probability.



With a small budget (50-100 queries) the per-class estimates are noisy and the deferral policy is close to never-defer. As the budget grows, the per-class expert accuracies converge to their true values and the deferral policy matches (or slightly exceeds) the Task-3 confidence policy without ever using the expert during training.

Task 5 — Interactive expert (optional)

A small Django interface presents one random test-set article per turn. A participant chooses a class, and the app records the submission together with the true label in the database (HumanExpertLabel) so it appears in the Django admin and survives across page reloads. A running accuracy is displayed as the participant labels more articles.

The purpose is not to run a real study but to demonstrate the plumbing: the same interface, plus the active-learning query strategy from Task 4, could be used to build a real human-in-the-loop labelling pipeline.

Software structure

The code is organised across proper Django-app files: ml.py (dataset loading + baseline), experts.py (Task 2), deferral.py (Task 3), active_learning.py (Task 4), models.py (Task 5 storage), forms.py (validation), and views.py as a thin controller. Unit tests live in tests.py.